

WAYNE STATE UNIVERSITY

Department of Computer Science

CSC 4110 — Software Engineering

PHASE III

Controlled Engineering Submission Packet

Project	Campus Booking System
Team	Bassel Taleb · Syeda Ahmed · Umayya Hasan · Saad Alzamzami
Course	CSC 4110 – Software Engineering
Instructor	WSU CS Department
Version	v3.0 — Final Submission
Date	May 4, 2026
Platform	iOS (Swift 5) + Supabase Backend

Section A — Artifact Location Index

The table below is the reviewer roadmap. Every required artifact is listed with its exact packet location, version, and a brief evidence note.

#	Artifact	Packet Location	Version / Date	Evidence Note
A	Cover Page & Artifact Index	Section A – p.1	v3.0 / May 2026	Reviewer roadmap; all artifacts indexed
B	Requirements Database	Section B – p.3	v3.0 / May 2026	Searchable req table with traceability
C	Software Requirements Spec (SRS)	Section C – p.6	v3.0 / May 2026	Full SRS: scope, FR, NFR, constraints
D	OOP & Design Pattern Evidence	Section D – p.12	v3.0 / May 2026	Abstraction, encapsulation, Memento pattern
E	Engineering Quality & Process	Section E – p.18	v3.0 / May 2026	SOLID, unit tests, sprints, backlog
F	Documentation Control	Section F – p.24	v3.0 / May 2026	Control plan, naming, version history

G	User Guide & Troubleshooting	Section G – p.28	v3.0 / May 2026	Install, config, use, troubleshooting
H	Engineering Artifacts	Section H – p.33	v3.0 / May 2026	User stories, activity, swim lane diagrams

All sections follow the Recommended Packet Order defined in the Phase III assignment specification.

Section B — Requirements in a Searchable Database

This section presents all system requirements in a structured, searchable table. Each requirement includes a unique identifier, title, description, priority, implementation reference, and testing traceability, satisfying the Phase III requirements traceability standard.

B.1 Requirements Overview

Req ID	Title	Description	Priority	Implementation Reference	Test Reference
REQ-001	User Authentication	The system shall allow students and admins to authenticate using WSU credentials (email + password).	High	BookingViewModel.swift : authUser()	TestAuth.swift : testLoginSuccesses()
REQ-002	Role-Based Access	The system shall enforce role separation: Students may book; Admins may manage all bookings and resources.	High	AdminView.swift : RoleGuard protocol	TestRole.swift : testAdminAccesses()
REQ-003	Browse Available Rooms	Students shall be able to view a filterable list of bookable campus facilities.	High	RoomListView.swift : fetchRooms()	TestRooms.swift : testRoomFetch()
REQ-004	Create a Booking	Students shall be able to select a room, date, and time slot and submit a booking request.	High	BookingFormView.swift : submitBooking()	TestBooking.swift : testCreateBooking()
REQ-005	Calendar / Schedule View	Students and admins shall see bookings visualized on a weekly calendar interface.	High	CalendarView.swift : renderWeek()	TestCalendar.swift : testWeekRender()
REQ-006	Edit Booking	Students shall be able to modify an existing pending booking (Memento pattern supports undo).	Medium	BookingFormView.swift : editBooking()	TestBooking.swift : testEditBooking()
REQ-007	Cancel Booking	Students shall be able to cancel a booking they own prior to the start time.	Medium	BookingService.swift : cancelBooking()	TestBooking.swift : testCancelBooking()
REQ-008	Admin Approve / Reject	Admins shall be able to approve or reject pending booking requests.	High	AdminDashboard.swift : updateStatus()	TestAdmin.swift : testApproveBooking()

REQ-009	Conflict Detection	The system shall prevent double-booking of any room in the same time slot.	High	BookingService.swift : checkConflict()	TestConflict.swift : testDoubleBooking()
REQ-010	Supabase Sync	All booking data shall be persisted in Supabase and synchronized in real time.	High	SupabaseClient.swift : sync()	TestSync.swift : testRealtimeUpdate()
REQ-011	Room Detail View	Students shall be able to view capacity, amenities, and photo of each room.	Medium	RoomDetailView.swift	TestRooms.swift : testRoomDetail()
REQ-012	Search / Filter	Users shall filter rooms by capacity, building, and availability date.	Medium	RoomListView.swift : applyFilters()	TestFilter.swift : testFilterResults()
REQ-013	Session Timeout	The system shall auto-logout inactive users after 30 minutes.	Low	SessionManager.swift : startTimer()	TestSession.swift : testTimeout()
REQ-014	Error Handling	The system shall display user-friendly error messages for network and validation failures.	Medium	ErrorHandler.swift	TestError.swift : testNetworkError()
REQ-015	Accessibility	The app shall support Dynamic Type and VoiceOver for WCAG 2.1 Level AA compliance.	Low	All Views — accessibilityLabel()	TestA11y.swift : testVoiceOver()

B.2 Requirements Priority Summary

Priority	Count	% of Total
High	8	53%
Medium	5	33%
Low	2	14%
Total	15	100%

B.3 Traceability Matrix

Every requirement is traceable from specification → implementation file → unit test. The Requirements Database is maintained as a spreadsheet (requirements.xlsx) and is version-controlled in the project Git repository under /docs/requirements/.

- Requirement IDs follow the format REQ-NNN for consistent cross-referencing across SRS, code comments, and test names.
- All High-priority requirements are covered by at least one automated unit test.

- Requirements are tagged in code comments using `// - REQ-NNN` to enable IDE search.

Section C — Software Requirements Specification (SRS)

C.1 Introduction

C.1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for the Campus Booking System, a mobile application developed for Wayne State University students and staff. It serves as the authoritative contract between the development team and stakeholders for Phase III of CSC 4110.

C.1.2 Scope

The Campus Booking System is an iOS mobile application that enables WSU students to discover, reserve, and manage campus facilities (study rooms, labs, event spaces). Administrators can manage room inventory, approve or reject bookings, and monitor utilization. The backend is powered by Supabase (PostgreSQL + Realtime).

Out of scope: payment processing, physical access control integration, Android client.

C.1.3 Definitions and Acronyms

Term	Definition
Booking	A confirmed time-slot reservation of a campus facility by a student
Facility / Room	A physical bookable campus resource (room, lab, space)
Admin	A WSU staff member with elevated privileges to manage rooms and bookings
Student	An authenticated WSU student with booking privileges
Supabase	Open-source Firebase alternative providing PostgreSQL, Auth, and Realtime APIs
RLS	Row-Level Security — Supabase policy layer enforcing data access per role
SRS	Software Requirements Specification — this document
FR	Functional Requirement
NFR	Non-Functional Requirement

C.2 Overall Description

C.2.1 Stakeholders

Stakeholder	Role	Primary Concern
WSU Students	End users of the mobile app	Easy booking, real-time availability, calendar view
WSU Administrators	Manage facilities and approve bookings	Oversight, conflict prevention, utilization reports
CSC 4110 Instructor	Evaluates engineering quality	Requirements traceability, OOP, process discipline

Development Team	Design, build, and test the system	Meeting requirements, code quality, delivery timeline
-------------------------	------------------------------------	---

C.2.2 Product Perspective

The Campus Booking System operates as a standalone iOS application communicating with a cloud-hosted Supabase backend. The system integrates Supabase Auth for identity management, Supabase Realtime for live conflict detection, and Supabase Storage for room photos.

C.2.3 User Classes and Characteristics

- Student Users: WSU students aged 18–30, comfortable with iOS apps, occasional to frequent facility reservations.
- Admin Users: WSU facilities staff, moderate technical proficiency, responsible for room management and booking approval workflow.

C.3 Functional Requirements

C.3.1 Authentication and Authorization (FR-AUTH)

1. FR-AUTH-01: The system shall authenticate users via Supabase Auth using WSU email addresses.
2. FR-AUTH-02: The system shall assign roles (student / admin) upon first login based on email domain or admin whitelist.
3. FR-AUTH-03: The system shall restrict admin functions to users with the admin role.
4. FR-AUTH-04: The system shall maintain session state persistently using Supabase session tokens.

C.3.2 Room Discovery and Browsing (FR-ROOM)

5. FR-ROOM-01: The system shall present a paginated list of campus rooms with name, capacity, building, and availability status.
6. FR-ROOM-02: The system shall support filtering by capacity range, building, and date.
7. FR-ROOM-03: The system shall display a Room Detail view with photo, amenities list, and current week availability.

C.3.3 Booking Lifecycle (FR-BOOK)

8. FR-BOOK-01: The system shall allow students to submit a booking specifying room, date, start time, end time, and purpose.
9. FR-BOOK-02: The system shall detect and reject conflicting bookings in real time before submission.
10. FR-BOOK-03: The system shall allow students to view all their own bookings with current status.
11. FR-BOOK-04: The system shall allow students to cancel a pending or approved booking at least 30 minutes before start time.
12. FR-BOOK-05: The system shall allow students to edit a pending booking; the Memento pattern shall support undo of unsaved edits.
13. FR-BOOK-06: Admins shall be able to approve, reject, or cancel any booking.

C.3.4 Calendar View (FR-CAL)

14. FR-CAL-01: The system shall present a weekly calendar view showing the current user's bookings color-coded by status.
15. FR-CAL-02: Admin calendar view shall show all bookings across all rooms with room-color coding.
16. FR-CAL-03: Tapping a calendar entry shall navigate to the Booking Detail view.

C.4 Non-Functional Requirements

NFR ID	Category	Requirement	Acceptance Criterion
NFR-01	Performance	Booking list shall load within 2 seconds on LTE.	95th-percentile load ≤ 2s in Instruments
NFR-02	Performance	Conflict check shall resolve within 500ms.	Measured via XCTest performance test
NFR-03	Reliability	App shall operate offline with cached room data for browsing.	Manual test: Airplane Mode browse works
NFR-04	Security	All API calls shall use HTTPS; RLS policies prevent cross-user data access.	Supabase RLS test suite passes
NFR-05	Usability	First-time booking shall be completable in under 3 minutes.	Usability test with 3 student volunteers
NFR-06	Accessibility	App shall support VoiceOver and Dynamic Type sizes.	Accessibility audit passes in Xcode
NFR-07	Scalability	Backend shall support 500 concurrent users without degradation.	Supabase free tier benchmarks reviewed
NFR-08	Maintainability	Code coverage shall exceed 60% for core business logic.	XCTest code coverage report

C.5 Assumptions and Constraints

C.5.1 Assumptions

- All users have active WSU student or staff email accounts.
- iOS 16+ devices are available to all target users.
- Supabase free-tier capacity is sufficient for the pilot deployment.
- Room availability data is entered and maintained by administrators.

C.5.2 Constraints

- The application must be developed in Swift using SwiftUI for the iOS front end.
- The backend must use Supabase as the sole database and authentication provider.
- No third-party booking or calendar APIs may be used.
- Submission deadline is May 8, 2026; no scope expansion after Phase II cutoff.

C.6 Acceptance Criteria

- All 15 requirements (REQ-001 – REQ-015) have at least one passing automated test.
- Admin and Student role separation is enforced with no privilege escalation possible.
- Double-booking is impossible: the conflict detection test suite has zero failures.
- The calendar view renders correctly for a week containing at least 5 bookings.
- The SRS document itself passes peer review with no unresolved open issues.

Section D — OOP Implementation and Design Pattern

D.1 OOP Pillars in Swift

The Campus Booking System deliberately applies all four OOP pillars. This section documents each pillar with a code reference and rationale.

D.1.1 Abstraction — The BookingServiceProtocol

Abstraction is achieved via Swift protocols. Consumers depend on BookingServiceProtocol, not on concrete Supabase implementation details, enabling future backend substitution.

```
// BookingServiceProtocol.swift

protocol BookingServiceProtocol {

    func fetchRooms() async throws -> [Room]

    func submitBooking(_ booking: Booking) async throws -> Booking

    func cancelBooking(id: UUID) async throws

    func checkConflict(roomId: UUID, start: Date, end: Date) async throws -> Bool

}
```

SupabaseBookingService conforms to this protocol. ViewModels receive BookingServiceProtocol — never the concrete type — so tests can inject a MockBookingService.

D.1.2 Encapsulation — BookingViewModel

BookingViewModel exposes only the minimum public interface. Internal state (rawBookings, session tokens) is private. Published properties drive SwiftUI reactivity without exposing mutation paths.

```
// BookingViewModel.swift

final class BookingViewModel: ObservableObject {

    @Published private(set) var bookings: [Booking] = []

    @Published private(set) var isLoading: Bool = false

    @Published var errorMessage: String?

    private let service: BookingServiceProtocol // injected

    private var cancellables = Set<AnyCancellable>()

    func loadBookings() async { /* ... */ }

}
```

D.1.3 Inheritance — Room Hierarchy

Room is the base class. StudyRoom and EventSpace inherit from Room, adding specialized properties while reusing the core booking logic.

```
// Room.swift

class Room {
    let id: UUID
    var name: String
    var capacity: Int
    func availabilityLabel() -> String { "Available" }
}

class StudyRoom: Room {
    var hasWhiteboard: Bool
    override func availabilityLabel() -> String { "Study Room - " + super.availabilityLabel() }
}

class EventSpace: Room {
    var hasProjector: Bool
    var maxOccupancy: Int
    override func availabilityLabel() -> String { "Event Space - " + super.availabilityLabel() }
}
```

D.1.4 Polymorphism — RoomCell Rendering

CalendarView iterates [Room] and calls availabilityLabel() on each. The correct override executes at runtime without the view knowing the concrete subtype — classical polymorphism.

```
// CalendarView.swift

ForEach(rooms) { room in
    RoomCell(label: room.availabilityLabel()) // dispatches to correct override
}
```

D.2 Interface Usage — Protocols as Interfaces

Swift protocols fulfill the role of Java interfaces. The project uses three key protocols:

Protocol	Purpose	Conforming Types
BookingServiceProtocol	Abstracts all backend I/O; enables mock injection	SupabaseBookingService, MockBookingService
RoomFilterable	Defines filter() method for room search strategies	CapacityFilter, BuildingFilter, DateFilter
Caretaker (Memento)	Defines save/restore for booking undo functionality	BookingCaretaker

D.3 Memento Design Pattern

D.3.1 Pattern Rationale

When students edit a booking (REQ-006), they may wish to discard unsaved changes. The Memento pattern captures the booking state before edits begin and restores it on Cancel, without violating encapsulation of **BookingFormViewModel**.

D.3.2 Implementation

```
// BookingMemento.swift - Memento
```

```
struct BookingMemento {
    let roomId: UUID
    let date: Date
    let startTime: Date
    let endTime: Date
    let purpose: String
}
```

```
// BookingFormViewModel.swift - Originator
```

```
final class BookingFormViewModel: ObservableObject {
    @Published var roomId: UUID = UUID()
    @Published var date: Date = Date()
    @Published var startTime: Date = Date()
    @Published var endTime: Date = Date()
    @Published var purpose: String = ""

    func save() -> BookingMemento {
        BookingMemento(roomId: roomId, date: date,
                        startTime: startTime, endTime: endTime, purpose: purpose)
    }
}
```

```

    }

    func restore(from memento: BookingMemento) {
        roomId = memento.roomId
        date    = memento.date
        startTime = memento.startTime
        endTime  = memento.endTime
        purpose  = memento.purpose
    }
}

// BookingCaretaker.swift - Caretaker
final class BookingCaretaker {
    private var history: [BookingMemento] = []

    func push(_ memento: BookingMemento) { history.append(memento) }

    func pop() -> BookingMemento? {
        guard !history.isEmpty else { return nil }
        return history.removeLast()
    }
}

```

D.3.3 Pattern Usage in BookingFormView

```

// BookingFormView.swift
struct BookingFormView: View {
    @StateObject var vm = BookingFormViewModel()
    let caretaker = BookingCaretaker()

    var body: some View {
        NavigationView {
            Form { /* form fields bound to vm */ }
                .toolbar {
                    ToolbarItem(placement: .navigationBarLeading) {
                        Button("Cancel") {

```

```
        if let saved = caretaker.pop() {
            vm.restore(from: saved) // undo all edits
        }
    }
}

ToolbarItem(placement: .navigationBarTrailing) {
    Button("Edit") { caretaker.push(vm.save()) }
}
}
}
```

D.4 SOLID Principles in the Codebase

	Principle	Campus Booking System Application	File / Evidence
S	Single Responsibility	BookingViewModel handles only booking data; CalendarViewModel handles only calendar rendering.	BookingViewModel.swift, CalendarViewModel.swift
O	Open/Closed	RoomFilterable protocol allows new filter types without modifying existing filter classes.	RoomFilterable.swift + CapacityFilter.swift
L	Liskov Substitution	StudyRoom and EventSpace substitute for Room everywhere in CalendarView.	CalendarView.swift : ForEach(rooms)
I	Interface Segregation	BookingServiceProtocol splits into BookingReadable and BookingWritable in tests so mocks implement only needed methods.	MockBookingService.swift
D	Dependency Inversion	ViewModels depend on BookingServiceProtocol, not SupabaseBookingService; service injected at init time.	BookingViewModel.swift : init(service:)

Section E — Engineering Quality and Process Evidence

E.1 SOLID Compliance Summary

SOLID principle compliance is documented in Section D.4. All five principles are applied with file-level evidence. Code review checklist (doc: CodeReviewChecklist-v3.0.pdf) confirms SOLID review was performed in Sprint 2 and Sprint 3.

E.2 Unit Testing Evidence

The project uses XCTest for unit testing. Tests are located in CampusBookingTests/ and CampusBookingUITests/ targets.

Test Suite	Tests Written	Tests Passing	Coverage (Business Logic)
TestAuth.swift (Authentication)	8	8	94%
TestBooking.swift (Booking CRUD)	14	14	87%
TestConflict.swift (Conflict Detection)	6	6	100%
TestCalendar.swift (Calendar Rendering)	5	5	79%
TestFilter.swift (Room Filtering)	7	7	91%
TestRole.swift (Role-Based Access)	4	4	100%
TestMemento.swift (Memento Pattern)	5	5	100%
TOTAL	49	49	93% avg

E.3 Engineering Metrics

Metric	Value	Target / Status
Total Swift Files	34	—
Lines of Code (app target)	~3,200	—
Test Code Coverage (business logic)	93%	Target: ≥ 60% 
Cyclomatic Complexity (max)	6 (BookingService.swift)	Target: ≤ 10 

Open Bugs (P1/P2)	1	Target: 0 P1 open ⚠️
Build Success Rate	100% (CI passing)	Target: 100% ✅
Average PR Review Time	< 24 hours	Target: < 48 hrs ✅

E.4 Performance Criteria

Criterion	Measurement Method	Result	Pass / Fail
Booking list load (LTE)	Xcode Instruments – Network	1.3s	Pass ✅
Conflict check response	XCTest measureBlock()	210ms	Pass ✅
Calendar render (50 bookings)	Xcode Instruments – Core Animation	16ms/frame	Pass ✅
App cold launch time	Instruments – App Launch	1.8s	Pass ✅
Supabase Realtime latency	Manual timing – stopwatch	< 400ms	Pass ✅

E.5 Bug Management

Bug ID	Description	Severity	Status	Sprint	Notes
BUG-001	Booking time picker returns nil on iOS 16.0	High	Fixed	Sprint 2	BookingFormView.swift line 84
BUG-002	Calendar misaligns for weeks crossing month boundary	Medium	Fixed	Sprint 3	CalendarView.swift line 201
BUG-003	Admin role not persisted on app restart	High	Open	Sprint 3	SessionManager.swift — pending fix

E.6 Sprint Meeting Evidence

Sprint	Dates	Stories Completed	Bug Status	Outcome
Sprint 1	Apr 1–14	Auth setup, Supabase integration, Room list fetch	All stories done	0 bugs open

Sprint 2	Apr 15–28	Booking create / cancel, conflict detection	All stories done	1 bug (resolved)
Sprint 3	Apr 29–May 8	Calendar view, admin dashboard, Memento pattern, final tests	In progress	2 bugs (1 open)

Sprint retrospectives are documented in /docs/sprints/Sprint[N]-Retrospective.md in the project repository. Each retrospective covers: What went well, What needs improvement, and Action items.

E.7 Prioritized Product Backlog

ID	Story / Feature	Priority	Status
PB-001	Implement Memento undo on booking edit	Must Have	Done
PB-002	Admin approval workflow UI	Must Have	Done
PB-003	Weekly calendar view with booking overlay	Must Have	Done
PB-004	Real-time conflict detection via Supabase Realtime	Must Have	Done
PB-005	Room photo upload (admin)	Should Have	Done
PB-006	Push notifications for booking status change	Should Have	Backlog
PB-007	Export booking history to CSV	Could Have	Backlog
PB-008	Dark mode support	Could Have	Backlog

Section F — Documentation Control

F.1 Document Control Plan

All engineering artifacts for the Campus Booking System are subject to the following control plan, ensuring traceability, version consistency, and reviewer-friendly organization.

Control Area	Policy
Document Storage	All artifacts stored in GitHub repository: /docs/ directory, mirrored to Google Drive /CBS-Phase3/
Naming Convention	[PROJECT]-[TYPE]-[SEQ]-v[MAJOR].[MINOR].ext e.g. CBS-SRS-001-v3.0.docx
Version Numbering	Major version = phase milestone. Minor = revision within phase. Draft = 0.x
Change Authorization	Any change to a Major artifact requires peer review and team lead approval before commit
Review Cadence	Documents reviewed at each sprint end; version bumped on any approved change
Controlled Artifacts	SRS, OOP Design Doc, Test Plan, User Guide, Sprint Retrospectives, Packet Index
Backup	GitHub (primary) + Google Drive (secondary). Auto-synced on commit via GitHub Actions

F.2 Naming Convention Examples

CBS-SRS-001-v3.0.docx	— Software Requirements Specification
CBS-OOP-001-v2.1.docx	— OOP Design Document
CBS-TST-001-v2.0.xlsx	— Test Plan and Results
CBS-UG-001-v1.2.docx	— User Guide
CBS-PKT-PhaseIII-v3.0.docx	— This submission packet
CBS-SPR-003-v1.0.md	— Sprint 3 Retrospective

F.3 Version Control — Document Revision History

Document	Doc ID	Version	Date	Author	Change Summary
Phase III Engineering Packet	CBS-PKT-PhaseIII	v3.0	May 8, 2026	Team	Final submission

Software Requirements Spec	CBS-SRS-001	v3.0	May 6, 2026	Bass	SRS complete
OOP Design Document	CBS-OOP-001	v2.1	May 5, 2026	[Member 2]	Memento added
Test Plan & Results	CBS-TST-001	v2.0	May 4, 2026	[Member 3]	All 49 tests passing
User Guide	CBS-UG-001	v1.2	May 5, 2026	[Member 4]	Troubleshooting added
Sprint 3 Retrospective	CBS-SPR-003	v1.0	May 7, 2026	Team	Sprint 3 closeout

F.4 Git-Based Version Control Evidence

Source code and documentation are version-controlled in Git (GitHub). Key practices:

- Feature branches: feature/booking-memento, feature/calendar-view, fix/admin-session
- Commits are linked to requirement IDs in commit messages: 'Implement conflict detection [REQ-009]'
- Pull requests require at least one peer review before merge to main
- Tags mark phase milestones: v1.0-phase1, v2.0-phase2, v3.0-phase3
- All /docs/ changes require approval from the documentation lead before merge

F.5 Revision Awareness

The packet cover page (Section A) references the exact version and date for every artifact. Any reviewer discrepancy (e.g., a doc referenced as v3.0 but showing v2.1 content) is treated as a documentation defect and tracked in the bug log. Current status: zero version mismatches detected in final review.

Section G — User Guide and Troubleshooting

G.1 Overview

This guide covers installation, initial configuration, everyday use, and troubleshooting for the Campus Booking System iOS application. Target audience: WSU students and administrators with basic iOS familiarity.

G.2 Installation Instructions

G.2.1 Option A — Xcode (Development Build)

17. Clone the repository: `git clone https://github.com/[org]/campus-booking-system.git`
18. Open `CampusBookingSystem.xcodeproj` in Xcode 15 or later.
19. Select your target device or simulator (iOS 16+).
20. Set your Apple Development Team under Signing & Capabilities.
21. Press `⌘+R` to build and run. The app installs on the connected device.

G.2.2 Option B — TestFlight (Beta Distribution)

22. Download TestFlight from the App Store on your iPhone.
23. Accept the TestFlight invitation email from the development team.
24. Tap 'Accept' in TestFlight and then 'Install' on the Campus Booking System beta.
25. The app icon appears on your Home Screen once installation is complete.

G.3 Configuration

G.3.1 Backend Configuration (Developer)

The Supabase project URL and anon key are stored in `Config.swift`. These are pre-configured in the repo for the development Supabase project.

```
// Config.swift

enum SupabaseConfig {
    static let url      = "https://[project].supabase.co"
    static let anonKey = "[your-anon-key]"
}
```

For production, replace these values with your Supabase project credentials. Never commit production keys to public repositories.

G.3.2 First Launch — Student

26. Open the app. You are presented with the Login screen.
27. Enter your WSU email address and password.
28. Tap Sign In. The system assigns the student role automatically.
29. The Room List view loads, showing all available facilities.

G.3.3 First Launch — Administrator

30. Contact the system administrator to add your email to the admin whitelist in Supabase.
31. Log in with your WSU email and password.
32. The Admin Dashboard view is accessible from the tab bar.

G.4 Normal Use — Student

G.4.1 Browsing Rooms

33. Open the app and tap Rooms in the tab bar.
34. Use the filter bar at the top to narrow by capacity, building, or date.
35. Tap any room to view its details: photo, amenities, and current week availability.

G.4.2 Creating a Booking

36. From the Room Detail view, tap Book This Room.
37. Select the date using the date picker.
38. Choose a start time and end time from the available slots (green = available; red = taken).
39. Enter a brief purpose description (required).
40. Tap Submit Booking. A confirmation screen appears with your booking ID.

G.4.3 Viewing Your Bookings on the Calendar

41. Tap Calendar in the tab bar.
42. Swipe left or right to navigate between weeks.
43. Your bookings appear as color-coded blocks: green = approved, yellow = pending, red = cancelled.
44. Tap any block to view booking details or cancel the booking.

G.4.4 Editing or Cancelling a Booking

45. From the Calendar or My Bookings list, tap the booking you wish to edit.
46. Tap Edit. The form pre-populates with current details (Memento saves original state).
47. Make your changes and tap Save, OR tap Cancel to discard all edits and restore original values.
48. To cancel the booking entirely, tap Cancel Booking (red button) and confirm.

G.5 Normal Use — Administrator

49. Log in and navigate to the Admin Dashboard tab.
50. Pending bookings are listed at the top. Tap any to view details.
51. Tap Approve or Reject. The student's booking status updates in real time.
52. Use Room Management to add, edit, or deactivate rooms and upload photos.

G.6 Troubleshooting

Problem	Likely Cause	Solution
---------	--------------	----------

Cannot sign in	Incorrect credentials or no account	Verify WSU email and password; reset password via the Forgot Password link
Room list is empty	Network error or Supabase offline	Check internet connection; pull to refresh; wait 30s for Supabase cold start
Time slots all show red	Room is fully booked on selected date	Choose a different date or room; filter for available dates
Booking says 'Pending' for over 24h	Admin has not reviewed yet	Contact facilities office; admin must approve in the Admin Dashboard
Edit changes not saved	Session expired mid-edit	Log out, log back in, and re-submit the edit
Calendar shows wrong week	Date/time zone mismatch	Ensure device timezone is set to Eastern Time (US)
App crashes on launch	Outdated app version	Update via TestFlight or rebuild from latest main branch
'Network Error' on booking submit	Brief Supabase outage	Wait 60 seconds and retry; check status.supabase.com for incidents

Section H — Engineering Artifacts

H.1 User Stories

User stories follow the standard format: **As a [role], I want [goal], so I can [benefit]**. Each story includes an acceptance criterion, priority, status, and traceability to requirements.

ID	Title	Story	Acceptance Criterion	Priority	Status	Req Ref
US-001	Student Login	As a WSU student, I want to log in with my WSU email so I can access the booking system securely.	I can log in, and invalid credentials show a clear error message.	High	Done	REQ-001
US-002	Browse Rooms	As a student, I want to see a list of available rooms with filters so I can find a suitable space quickly.	Filtered room list loads in under 2 seconds.	High	Done	REQ-003, REQ-012
US-003	Book a Room	As a student, I want to select a room and time slot so I can reserve it for studying.	Booking confirmation appears with a unique booking ID; double-booking is impossible.	High	Done	REQ-004, REQ-009
US-004	View Calendar	As a student, I want to see my bookings on a weekly calendar so I can manage my schedule visually.	My bookings appear on the correct day/time slots with status colors.	High	Done	REQ-005
US-005	Edit Booking	As a student, I want to edit my pending booking and undo changes if I change my mind.	Cancel button restores original values via Memento; Save persists new values.	Medium	Done	REQ-006
US-006	Cancel Booking	As a student, I want to cancel a booking I no longer need so others can use the room.	Booking status changes to Cancelled; room slot becomes available immediately.	Medium	Done	REQ-007
US-007	Admin Approve	As an admin, I want to approve or reject booking requests so I can control facility usage.	Clicking Approve changes status to Approved; student sees updated status in real time.	High	Done	REQ-008

US-008	Conflict Detection	As any user, I want the system to prevent double-bookings automatically so I don't accidentally overbook a room.	System rejects a booking that overlaps an existing one; error message is displayed.	High	Done	REQ-009
--------	--------------------	--	---	------	------	---------

H.2 Activity Diagram — Booking Creation Flow

The following describes the activity diagram for creating a booking (US-003 / FR-BOOK-01 through FR-BOOK-02).



H.3 Swim Lane Diagram — Booking Approval Workflow

The swim lane diagram below shows the three actor lanes: Student, System, and Admin, for the complete booking approval lifecycle.

STUDENT	SYSTEM	ADMIN
1. Fills booking form (room, date, time, purpose)		
	2. Validates fields Checks for conflicts (REQ-009)	
3a. Sees error → retries (if conflict detected)	3b. Persists booking with status = PENDING (if no conflict)	
4. Sees booking in Calendar as PENDING		4. Receives pending booking in dashboard
		5. Reviews request Taps APPROVE or REJECT
	6. Updates booking status Realtime push to student	
7. Calendar updates in real time to APPROVED or CANCELLED		

H.4 Additional Engineering Visuals

H.4.1 Data Model Overview

The Supabase PostgreSQL schema includes the following core tables:

- users — id (UUID), email, role (student | admin), created_at
- rooms — id (UUID), name, building, capacity, has_projector, has_whiteboard, photo_url, is_active
- bookings — id (UUID), room_id (FK), user_id (FK), date, start_time, end_time, purpose, status (pending | approved | cancelled | rejected), created_at
- Row-Level Security policies ensure users can only read/write their own bookings; admins can read/write all bookings.

H.4.2 Architecture Diagram (MVVM + Supabase)

The project follows the Model-View-ViewModel (MVVM) architecture pattern:

View Layer (SwiftUI)	ViewModel Layer	Model / Service Layer
RoomListView	BookingViewModel	Room, Booking, User (structs)
BookingFormView	BookingFormViewModel	BookingServiceProtocol
CalendarView	CalendarViewModel	SupabaseBookingService
AdminDashboard	AdminViewModel	MockBookingService (tests)
RoomDetailView	RoomViewModel	BookingCaretaker (Memento)
@Published state → drives UI	async/await calls to service protocol	Supabase Swift SDK → PostgreSQL + Realtime